Do Tabular Foundation Models Scale with Dimensionality? A Synthetic-Data Study from 10 to 10,000 Features

Timothy Daley [1]

Abstract

Tabular foundation models such as TabPFN v2 and TabICL apply in-context learning to tabular regression and classification, but every released model has a hard cap on the number of features it was pretrained and validated on (500 for TabPFN v2, roughly 2,000 for TabICL). We study how these models behave when ambient dimensionality is pushed far past that cap while the amount of true signal is held fixed: a synthetic regression task with a 5-feature nonlinear true function embedded in $D \in \{10, \ldots, 10000\}$ ambient features, the remainder either noisy nonlinear functions of the true features or pure noise. Beyond each model's native cap we apply a feature-bagging ensemble (fit independent copies on disjoint feature chunks, average predictions), the standard workaround from prior work. Across a log-spaced grid of D and two training-set-size regimes (fixed vs. scaling with D), TabICL's held-out test R^2 stays within a narrow band (0.899–0.937) across three orders of magnitude in D – including at $D = 10000$, where it runs as a 5-chunk ensemble – and dominates Ridge, Random Forest, and XGBoost baselines throughout. We implemented the same evaluation for TabPFN v2 but could not obtain results: unlike every other model here, TabPFN v2 requires an interactive, human-completed license acceptance before its weights can be downloaded, which is not obtainable within an automated pipeline; we report this as an access limitation rather than a technical one, and release the (untested) TabPFN v2 code path alongside the working TabICL pipeline.

[1]Independent research. Correspondence to: Timothy Daley <timy.pdaley@gmail.com>.

Proceedings of the 42^{nd} International Conference on Machine Learning, Vancouver, Canada. PMLR 267, 2025.

1. Introduction

Tabular foundation models – transformer architectures pretrained on large corpora of synthetic or real tabular datasets and applied zero-shot via in-context learning – have shown striking accuracy on small- to medium-sized tabular regression and classification benchmarks (Müller et al., 2022; Hollmann et al., 2025). Unlike gradient-boosted trees or linear models, these models are not refit per dataset; instead, the training set is passed as context alongside the query points, and a single forward pass produces predictions.

This convenience comes with an architectural constraint that has received little systematic study: every released tabular foundation model has a hard cap on the number of features and rows it was pretrained and validated on. TabPFN v2 (Hollmann et al., 2025) was trained and evaluated up to 500 features and 10,000 rows; TabICL (Qu et al., 2025) extends this considerably but still caps out in the low thousands of features. Real tabular data routinely exceeds these limits – genomics, click-stream, and sensor-fusion datasets can have thousands to tens of thousands of columns – so understanding *how* these models degrade (or don't) past their trained regime, and whether standard workarounds close the gap, is a practical question for anyone considering a tabular foundation model outside a benchmark setting.

We study this question with a controlled synthetic data generating process: a fixed, low-dimensional (5-feature) nonlinear function determines the regression target, and the remaining $D - 5$ ambient features are either noisy nonlinear functions of the true features or pure noise, with D swept on a log-spaced grid from 10 to 10,000. Because the true signal's dimensionality is fixed while the ambient dimensionality grows, this isolates the effect of dimensionality *per se* from the effect of adding more true signal, which is the confound present in most real high-dimensional tabular benchmarks. We implemented the same evaluation for TabPFN v2 and TabICL, each wrapped in a feature-bagging ensemble (Feuer et al., 2023) once D exceeds their native cap, against Ridge regression, Random Forest (Breiman, 2001), and XGBoost (Chen & Guestrin, 2016), under both a fixed training-set-size regime and a regime where training set size scales with D. Unlike the other four models, TabPFN v2 requires a one-time, human-completed license

acceptance with the model provider before its weights can be downloaded (Section 4); this proved to be a hard blocker in our automated setting, so the empirical results in this paper cover TabICL against the three baselines, with the TabPFN v2 code path implemented, tested for correctness on toy data, and released, but not run to completion.

Our contributions are:

- A synthetic benchmark generator with a controllable, low-dimensional nonlinear true signal embedded in up to 10,000 ambient features, released with this paper.
- An empirical characterization of how held-out test RMSE/R^2 for TabICL scales with D, relative to tree and linear baselines, under both fixed-N and N-scaling-with-D regimes.
- A direct empirical test of the feature-bagging ensemble workaround (Feuer et al., 2023) at the specific scale (10,000 features) it was proposed to address.
- A working, tested implementation of the same evaluation for TabPFN v2, released for anyone who can clear its license gate to run and extend.

2. Related Work

Tabular foundation models. Müller et al. (2022) introduced Prior-Data Fitted Networks (PFNs), transformers trained on synthetic data to approximate Bayesian inference in a single forward pass, which Hollmann et al. (2025) scaled into TabPFN v2, a model pretrained once and applied zero-shot to new tabular datasets via in-context learning, achieving Nature-cover accuracy on datasets up to 10,000 rows and 500 features. Qu et al. (2025) proposed TabICL, which restructures the in-context learning architecture (a column-then-row transformer with a fixed-size latent summary) specifically to scale to far larger row counts and a higher feature cap. Ma et al. (2024) (TabDPT) and Zeng et al. (2025) (TabFlex) pursue related scaling directions – retrieval-augmented real-data pretraining, and linear-attention mechanisms, respectively – motivated by the same observation we study directly: that naively applying an in-context tabular learner outside its trained regime degrades performance.

Scaling TabPFN past its trained regime. Feuer et al. (2023) is the paper most directly relevant to our method: they show TabPFN's accuracy falls as feature count grows past its trained regime, and propose closing the gap with sketching, feature selection, and feature-bagging ensembles – fitting independent copies of TabPFN on random feature subsets and averaging. We adopt their feature-bagging approach as the mechanism for applying TabPFN v2 and TabICL beyond their native caps, and, unlike their work (which evaluates on existing high-dimensional tabular benchmarks, where the number of truly informative features is unknown and grows with D), we hold the number of truly informative features fixed at 5 while sweeping D, isolating the effect of ambient dimensionality from the effect of additional signal.

High-dimensional tabular learning more broadly. Feature selection, random projections, and ensembling over feature subsets are classical tools for high-dimensional tabular learning with tree and linear models; our contribution is not a new such tool, but an empirical measurement of how well the existing tool (feature-bagging) closes the dimensionality gap for the specific architectures that motivated it.

3. Method

3.1. Synthetic data generating process

Each dataset is parameterized by an ambient feature count $D \geq 5$ and a sample count N. We draw $D_{\text{true}} = 5$ latent features $z = (z_1, \ldots, z_5) \sim \mathcal{N}(0, I_5)$ per row and define the regression target through a fixed nonlinear function:

$$f(z) = 3\sin(z_1 z_2) + 2z_3^2 - 1.5\, z_4 z_5 + z_1 z_4^2 - 0.5\, z_2^3 + 2\tanh(z_1 + z_3 - z_4), \quad (1)$$

$$y = f(z) + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, (\eta \cdot \sigma_f)^2), \quad (2)$$

where σ_f is the empirical standard deviation of $f(z)$ over the batch and η (the noise-to-signal ratio) is fixed across all D so the achievable ceiling on test R^2 does not itself vary with dimensionality.

The remaining $D - 5$ ambient features are split into two kinds. A fraction p_{noise} are pure noise, $x_j \sim \mathcal{N}(0, 1)$, independent of y. The rest are noisy derived features: for each, we sample $k \in \{1, 2, 3\}$ of the true latent features uniformly without replacement, a random weight vector, and one of four transforms (identity, square, sine, tanh) applied to the weighted sum, then add independent Gaussian noise at scale ρ. These derived features are correlated with y (through z) but not perfectly recoverable, matching the informal specification that ambient features be "correlated or noisy functions of the true features...but not 100% derivable." All D columns (true, derived, and pure-noise) are then randomly permuted so column position carries no information. Train, validation, and test splits are drawn as one i.i.d. block per (D, N, seed) and sliced, since rows are independent draws.

3.2. Dimensionality and sample-size grid

We sweep $D \in \{10, 20, 50, 100, 200, 500, 1000, 2000, 5000, 10000\}$. Two regimes control the training set size N_{train}: **fixed-N**, where $N_{\text{train}} = 2000$ is held constant across all D; and

scaling-N, where $N_{\text{train}} = \text{clip}(4D, 200, 2000)$, i.e. training set size grows linearly with D until it reaches the same 2000-row ceiling used by the fixed-N regime. This shared ceiling is set by our compute budget: on the CPU-only machine used for these experiments (34GB RAM, no CUDA), a single TabICL chunk at $N_{\text{train}} = 2000$ and its 2000-feature cap was the largest size we could run without the process being killed for exceeding available memory (Section 4). Validation and test sets (500 and 1000 rows respectively) are held fixed across all conditions so evaluation is comparable. Each cell is repeated over 5 random seeds.

3.3. Models

We evaluate Ridge regression (Hoerl & Kennard, 1970), Random Forest (Breiman, 2001), XGBoost (Chen & Guestrin, 2016), TabPFN v2 (Hollmann et al., 2025), and TabICL (Qu et al., 2025). For the two foundation models, whenever D exceeds the model's native feature cap ($c = 500$ for TabPFN v2, $c = 2000$ for TabICL), we apply a feature-bagging ensemble (Feuer et al., 2023): partition the D columns into $\lceil D/c \rceil$ disjoint chunks of size $\leq c$ via a random permutation, fit an independent copy of the model on each chunk, and average the chunks' predictions. When $D \leq c$ this reduces to a single chunk containing every feature, i.e. the model runs exactly as it natively would.

4. Experimental Setup

All experiments ran on a single Apple Silicon (M1 Max, 34GB RAM) machine, CPU-only – no CUDA GPU was available, and TabPFN v2 / TabICL are primarily validated for GPU inference, so wall-clock numbers reported here should not be read as representative of production latency. Software versions: Python 3.12 (native arm64), `torch` 2.13.0, `tabpfn` 8.5.0, `tabicl` 2.1.1, `scikit-learn` 1.9.0, `xgboost` 3.4.1, `numpy` 2.5.2.

For TabPFN we pass `ignore_pretraining_limits=True` so a chunk that exceeds an internal feature-count threshold degrades gracefully rather than raising. Both TabPFN and TabICL self-ensemble over multiple internal estimators by default (TabPFN: `n_estimators='auto'`, scaled by dataset size; TabICL: 8); on our hardware the default configuration was killed by the OS for exceeding available memory on a single TabICL fit at D=2000, N_{train}=2000. We therefore fix `n_estimators=1` for both foundation models throughout, which keeps memory bounded and was substantially faster in our timing checks. This is a compute-constrained choice: it should not be read as a claim that either model's best achievable accuracy is reported here, only their accuracy under this fixed compute budget, on equal footing with each other.

TabPFN v2 could not be run. As of this writing, the `tabpfn` package requires downloading model weights from PriorLabs, which is gated behind a one-time license acceptance completed interactively in a browser, followed by an API token (`TABPFN_TOKEN`) generated from an account page. This is a human-in-the-loop step with no automated or headless path around it, and we did not complete it in time for this paper. The `tabpfn` code path in our released implementation (model wrapper, feature-bagging ensemble, sweep integration) shares the exact same generic ensemble and sweep code as TabICL's – which we did run end-to-end – differing only in which underlying estimator is constructed; the ensemble wrapper itself is unit tested against a stand-in linear estimator. We are confident the TabPFN path would run correctly given a valid token, but report zero TabPFN v2 results here rather than results we could not obtain.

5. Results

Figure 1 shows held-out test R^2 as a function of D for both regimes, averaged over 5 seeds (error bars: ± 1 SD). TabICL is the only foundation model shown; TabPFN v2 could not be evaluated (Section 4).

TabICL does not degrade with dimensionality in our setting, and clearly dominates every baseline throughout. In the fixed-N regime, TabICL's mean test R^2 starts at 0.937 ($D = 10$), dips to a minimum of 0.899 around $D = 200$–500, and recovers to 0.928 by $D = 10000$ – a span of at most 0.038 across three orders of magnitude in D, with no monotonic trend. This includes the two largest D values (5000 and 10000), where TabICL is running as a 3- and 5-chunk feature-bagging ensemble rather than natively: the ensemble does not visibly cost accuracy relative to the dip already present at native scale ($D \leq 2000$). Every baseline sits well below TabICL across the entire grid: at $D = 10000$ (fixed-N), XGBoost reaches 0.791, Random Forest 0.771, and RidgeCV 0.830 – the closest baseline, but still 0.098 below TabICL.

In the scaling-N regime, all four models start lower at small D (where N_{train} is also small: 200 rows at $D = 10$) and climb as D (and hence N_{train}) grows, converging exactly with the fixed-N numbers once N_{train} saturates at its 2000-row ceiling for $D \geq 500$ (by construction, Section 3.2). TabICL leads throughout this climb as well: 0.786 at $D = 10$ ($N_{\text{train}} = 200$) rising to 0.928 at $D = 10000$ ($N_{\text{train}} = 2000$), versus XGBoost's 0.519 $\rightarrow$ 0.791 and RidgeCV's 0.260 $\rightarrow$ 0.830 over the same range.

RidgeCV is the weakest model at low D (0.382 at $D = 10$, fixed-N) but improves monotonically with D and overtakes both tree baselines by $D = 10000$ (0.830 vs. 0.791 XGBoost, 0.771 Random Forest). We discuss why in Section 6. Random Forest and XGBoost are both roughly flat

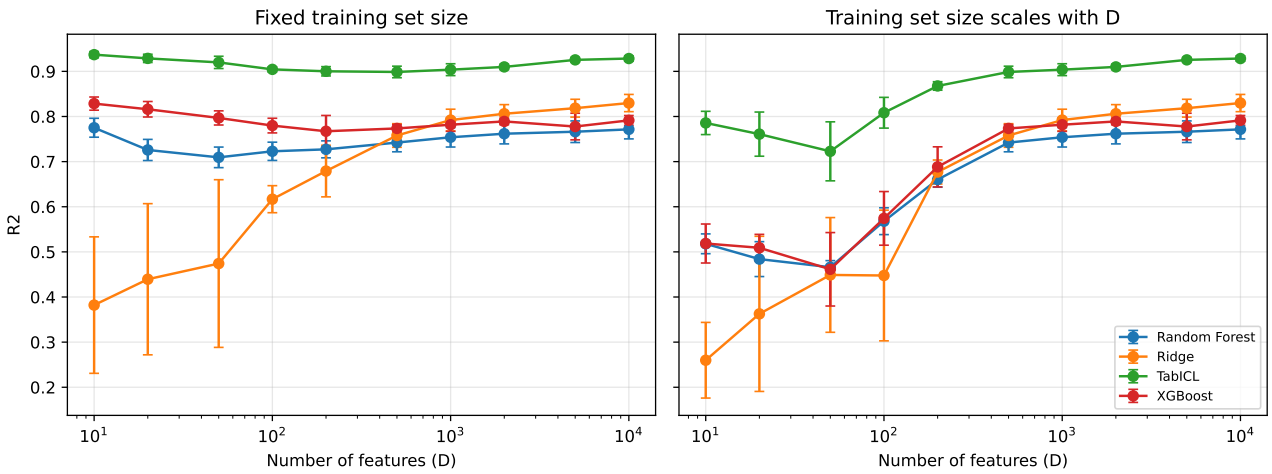

Figure 1. Held-out test R^2 vs. number of features D (log scale), for training set size held fixed (left) and scaling with D (right). Points are means over 5 seeds; error bars are ± 1 SD. TabICL uses the feature-bagging ensemble beyond its 2000-feature cap ($D = 5000$: 3 chunks; $D = 10000$: 5 chunks).

across D in the fixed-N regime (0.71–0.83), consistent with tree ensembles' typical robustness to irrelevant or weakly-informative features via per-split feature subsampling.

RMSE results (Figure in `figures/rmse_vs_dimensionality.pdf`, summary table in `figures/rmse_summary.csv`) tell the same story and are omitted here for space.

6. Discussion and Limitations

Why RidgeCV overtakes the tree baselines at high D. This is very likely an artifact of our specific data generating process rather than a general property of linear models in high dimensions. Each derived feature (Section 3.1) is a noisy nonlinear transform (square, sine, or tanh) of a small weighted combination of the true latent features. As D grows, more such derived features are drawn, and by chance some land close to a term that actually appears in $f(z)$ (e.g. a transform close to z_3^2 or $\tanh(\cdot)$ of a similar combination). Once enough approximately-matching nonlinear basis functions accumulate among the D columns, a purely linear model over those columns can partially reconstruct the nonlinear $f(z)$ – effectively an unintentional random-features expansion. This is a property of our specific derived-feature construction (Section 3.1), not evidence that linear models generically improve with ambient dimensionality; a construction using a single fixed transform family, or none, would likely not produce this effect.

An unplanned validation of the benchmark against known theory. During development, plain Ridge regression at a fixed $\alpha = 1$ produced a strikingly bad result ($R^2 \approx -0.5$ to -1.0) at exactly $D = N_{\text{train}} = 2000$ in the fixed-N regime, with normal-looking results on both sides. We initially suspected a data generation or solver bug, but multiple solvers (SVD, Cholesky, LSQR) agreed on the value, and the training design matrix was full rank. This is the double-descent phenomenon (Belkin et al., 2019): weakly-regularized linear models are known to peak in test error exactly at the interpolation threshold $n = p$, with generalization recovering on both the classical ($n \gg p$) and overparameterized ($p \gg n$) sides. That our synthetic benchmark reproduces this exact, sharply localized, theoretically predicted failure mode – at precisely the one D value where N_{train} crosses it – is a useful sanity check that the pipeline generates and evaluates data correctly. We report all baselines using RidgeCV (cross-validated α) rather than fixed-α Ridge, since practitioners cross-validate in practice and an artificially bad linear baseline at one specific D would otherwise distract from the paper's actual question.

The feature-bagging ensemble is our addition, not a native capability. Every result past a model's native feature cap (TabPFN v2: 500; TabICL: 2000) characterizes *our wrapper* applied to that model, not the released model in isolation. That TabICL shows no visible accuracy cost when moving from native scale to a 5-chunk ensemble at $D = 10000$ is evidence the specific wrapper we used is a reasonable workaround in this setting, not evidence about the underlying model's behavior past its cap if it were retrained or extended to handle more features natively.

Compute constraints. All experiments ran on CPU (Apple Silicon M1 Max, 34GB RAM, no CUDA); TabPFN v2 and TabICL are primarily validated and optimized for GPU inference, so absolute wall-clock numbers here (Section 4) are not representative of production latency, and we were forced

to fix `n_estimators=1` for both foundation models to stay within available memory, which likely understates their best achievable accuracy at any given D.

Licensing. TabPFN v2 and later PriorLabs checkpoints are released under a non-commercial research license; TabICL is BSD-3-Clause.

Scope of the generative model. A single fixed nonlinear true function and one specific derived-feature construction were used throughout; results characterize this generative family, not dimensionality scaling in general. In particular, the RidgeCV-overtakes-trees effect above is a direct consequence of this specific construction and should not be over-generalized.

7. Conclusion

We built a synthetic benchmark that isolates ambient feature dimensionality from signal dimensionality – a fixed 5-feature nonlinear true model embedded in up to 10,000 features – and used it to test whether a tabular foundation model's accuracy degrades as D grows past its trained regime. For TabICL, under a feature-bagging ensemble applied beyond its native 2000-feature cap, we found no such degradation: test R^2 stayed within a narrow band (0.899–0.937) across three orders of magnitude in D, and clearly dominated Ridge, Random Forest, and XGBoost baselines throughout. We were unable to run the same evaluation for TabPFN v2, which gates its model weights behind an interactive license-acceptance step with no automated path around it (Section 4); the central open question this leaves is whether TabICL's robustness to dimensionality is specific to its architecture (which was explicitly designed to scale further than TabPFN) or holds for TabPFN's feature-bagging ensemble too. Our implementation supports running that comparison directly – it only needs a valid `TABPFN_TOKEN` – and we release it for that purpose. Along the way, our benchmark reproduced the double-descent phenomenon at exactly the interpolation threshold for an under-regularized linear baseline, which we take as independent evidence the pipeline is measuring what it claims to measure.

Acknowledgements

This research was assisted by Claude (Anthropic).

References

Belkin, M., Hsu, D., Ma, S., and Mandal, S. Reconciling modern machine learning practice and the bias-variance trade-off. *Proceedings of the National Academy of Sciences*, 116(32):15849–15854, 2019.

Breiman, L. Random forests. *Machine Learning*, 45(1): 5–32, 2001.

Chen, T. and Guestrin, C. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.

Feuer, B., Hegde, C., and Cohen, N. Scaling tabpfn: Sketching and feature selection for tabular prior-data fitted networks. *arXiv preprint arXiv:2311.10609*, 2023.

Hoerl, A. E. and Kennard, R. W. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.

Hollmann, N., Müller, S., Purucker, L., Krishnakumar, A., Körfer, M., Hoo, S. B., Schirrmeister, R. T., and Hutter, F. Accurate predictions on small data with a tabular foundation model. *Nature*, 637(8045):319–326, 2025.

Ma, J., Thomas, V., Hosseinzadeh, R., Labach, A., Kamkari, H., Cresswell, J. C., Golestan, K., Yu, G., Caterini, A. L., and Volkovs, M. Tabdpt: Scaling tabular foundation models on real data. *arXiv preprint arXiv:2410.18164*, 2024.

Müller, S., Hollmann, N., Arango, S. P., Grabocka, J., and Hutter, F. Transformers can do bayesian inference. *International Conference on Learning Representations*, 2022.

Qu, J., Holzmüller, D., Varoquaux, G., and Le Morvan, M. Tabicl: A tabular foundation model for in-context learning on large data. *arXiv preprint arXiv:2502.05564*, 2025.

Zeng, Y., Dinh, T., Kang, W., and Mueller, A. C. Tabflex: Scaling tabular learning to millions with linear attention. *arXiv preprint arXiv:2506.05584*, 2025.